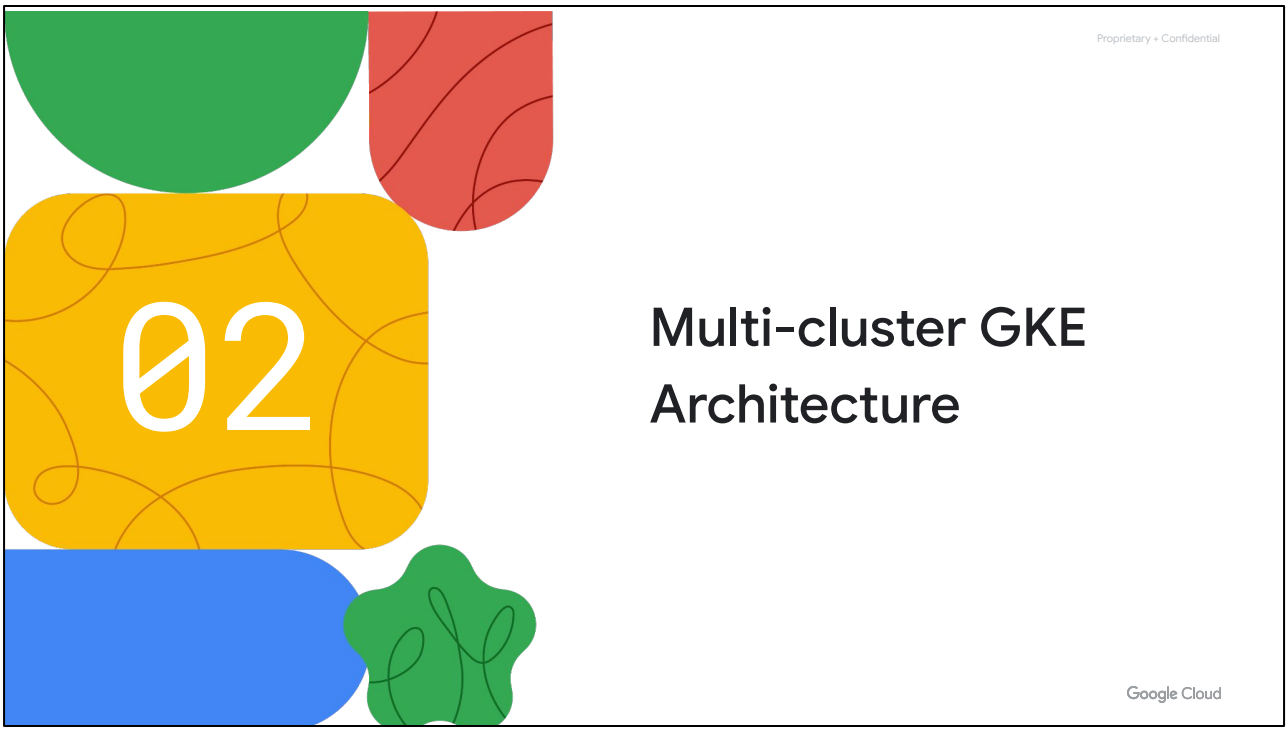
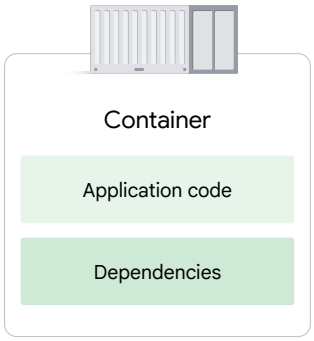


Multi-cluster GKE infrastructure provides a centrally managed, highly available, and secure platform for running your containerized workloads at scale.

Before exploring the architecture and features of multi-cluster GKE, let's begin with a review of single cluster Google Kubernetes Engine and containers.

Proprietary + Confidential

Review: containers



- Portable, standalone, executable packages of software.
- Contain everything needed to run an application.
- Can be leveraged by DevOps teams to rapidly deploy and scale applications.

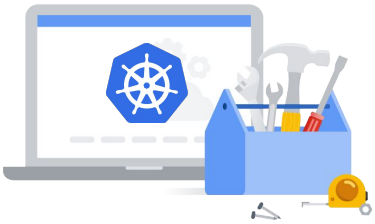
Google Cloud

Containers are portable, standalone, executable packages of software that include everything needed to run an application: code, runtime, system tools, system libraries, and settings.

DevOps teams can leverage containers to rapidly deploy and scale applications.

Proprietary + Confidential

Review: Kubernetes



Kubernetes provides PaaS features like:

- Automated placement
- Service discovery and load balancing
- Automated rollouts and rollbacks
- Resource management
- Health monitoring and healing
- Centralized management

Google Cloud

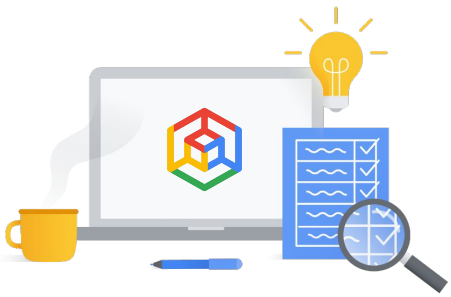
Managing containers at scale is complex and involves tasks like placement, traffic distribution, and release management.

Kubernetes automates container management tasks by providing Platform-as-a-service features like:

- Automated placement
- Service discovery and load balancing
- Automated rollouts and rollbacks
- Resource management
- Health monitoring and healing
- And centralized management for containerized applications

Proprietary + Confidential

Review: GKE



- A fully-managed service that handles infrastructure provisioning, networking, load balancing, security, and upgrades
- User friendly console, scalability, high availability, security, and cost-effectiveness

Google Cloud

Google Kubernetes Engine, or GKE, is a fully managed service that handles infrastructure provisioning, networking, load balancing, security, and upgrades. GKE offers a user-friendly console, scalability, high availability, security, and cost-effectiveness.

Proprietary + Confidential

Multi-cluster GKE Architecture

01 Centralized cluster management

02 Multi-cluster GKE Architecture

03 Create, connect, and manage GKE fleet clusters

04 Access GKE fleet clusters

Google Cloud

Now that you've reviewed GKE and containers, let's transition to scaling GKE to multi-cluster architectures. Understanding the difference in a scaled architecture is critical for leveraging GKE capabilities and optimizing your multi-cluster scaled deployments.

So, in this section of this course titled "Multi-cluster GKE Architecture", you'll learn to:

- Recognize how GKE can be used to centralize cluster management for multi-cluster environments.
- Examine the architecture of Multi-cluster GKE
- Create, connect, and manage GKE fleet clusters.
- Securely access GKE fleet clusters.

Proprietary + Confidential

Multi-cluster GKE Architecture

01 Centralized cluster management

02 Multi-cluster GKE Architecture

03 Create, connect, and manage GKE fleet clusters

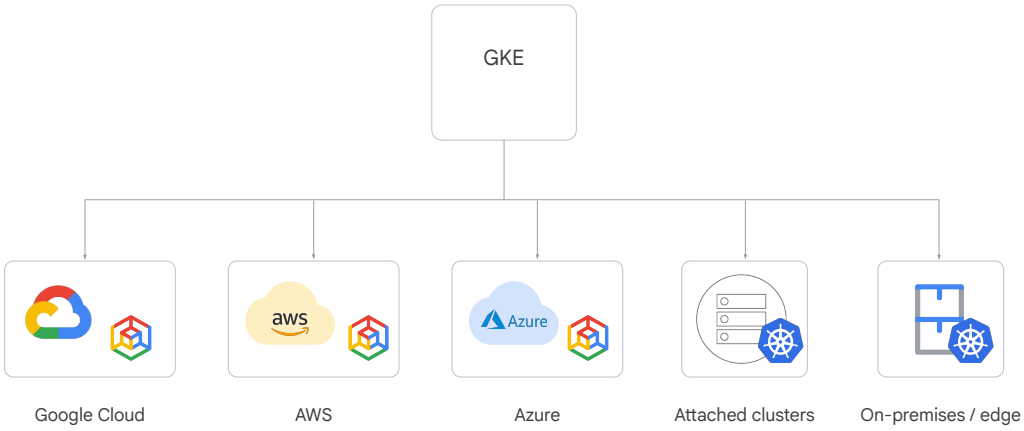
04 Access GKE fleet clusters

Google Cloud

GKE is a tool to help you manage your clusters that are running in Google Cloud, but GKE can also assist with clusters running across multiple providers, offering consistent configuration, security, and management.

Proprietary + Confidential

Manage clusters anywhere with GKE

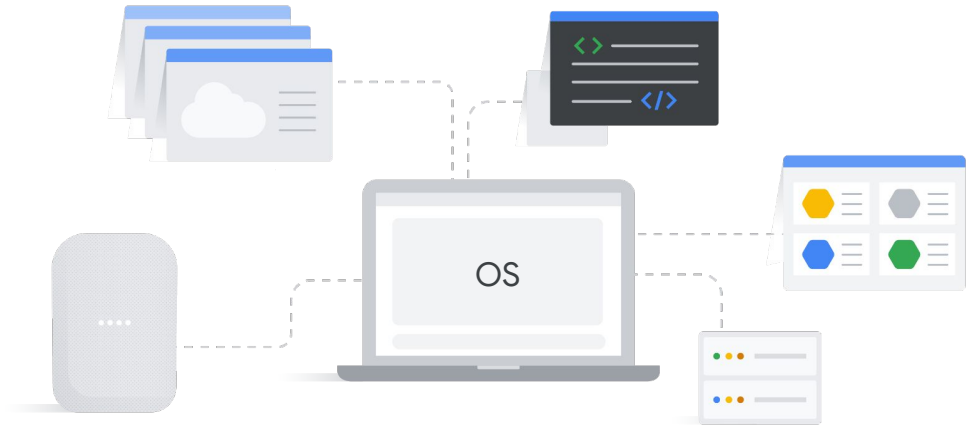


Google Cloud

With GKE, you can manage your clusters in one place, regardless of where they're located. From running clusters in Google Cloud, to Azure AKS or AWS EKS, GKE provides a comprehensive, centralized, and fully-managed Kubernetes experience. It can even be used for on-premises environments and edge locations, but this course focuses on hybrid and multi-cloud configurations.

Proprietary + Confidential

GKE provides centralized management



Google Cloud

Think of an operating system. It provides a consistent interface and set of tools for managing different hardware and software components.

Proprietary + Confidential

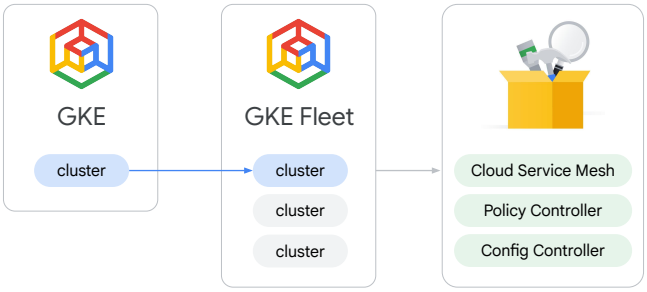
GKE provides centralized management

Google Cloud

Similarly, GKE acts as an operating system for your Kubernetes clusters, providing a unified platform for managing and configuring them consistently across different environments.

Proprietary + Confidential

Register clusters with a GKE fleet



Use Workload Identity for registration.

Google Cloud

To benefit from additional features like Cloud Service Mesh, Policy Controller and Config Controller, you must register your GKE clusters with a GKE fleet.

GKE clusters can leverage the Connect gateway with your Google ID to authenticate to clusters, with support for third party identity providers using workforce identity federation, and with group-based authentication support via GKE Identity Service.

Proprietary + Confidential

Multi-cluster GKE Architecture

01 Centralized cluster management

02 Multi-cluster GKE Architecture

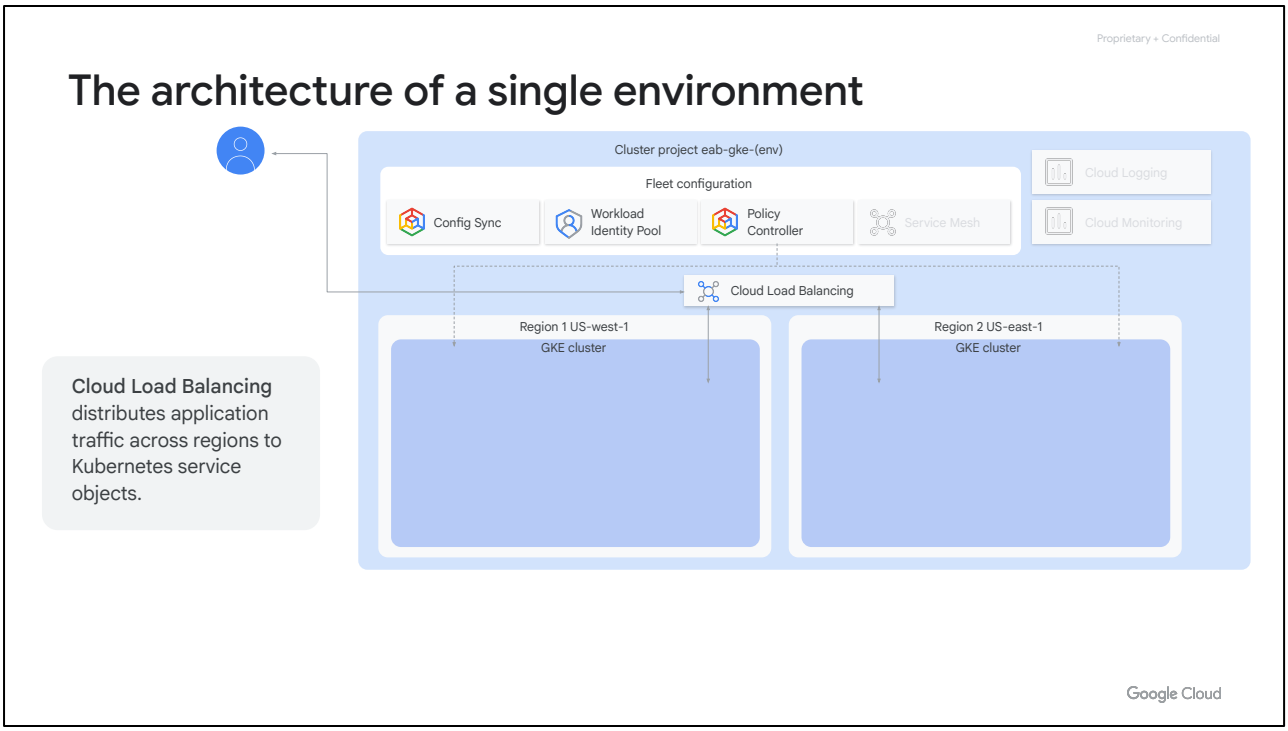
03 Create, connect, and manage GKE fleet clusters

04 Access GKE fleet clusters

Google Cloud

GKE provides a robust infrastructure for deploying developer platforms. Let's explore the key elements that form this infrastructure when building multi-cluster environments.

Enterprise infrastructure can be complex, so let's start with a small example and scale it up to a size that would use the benefits of GKE.



Let's begin by examining the structure of a single environment for developers.

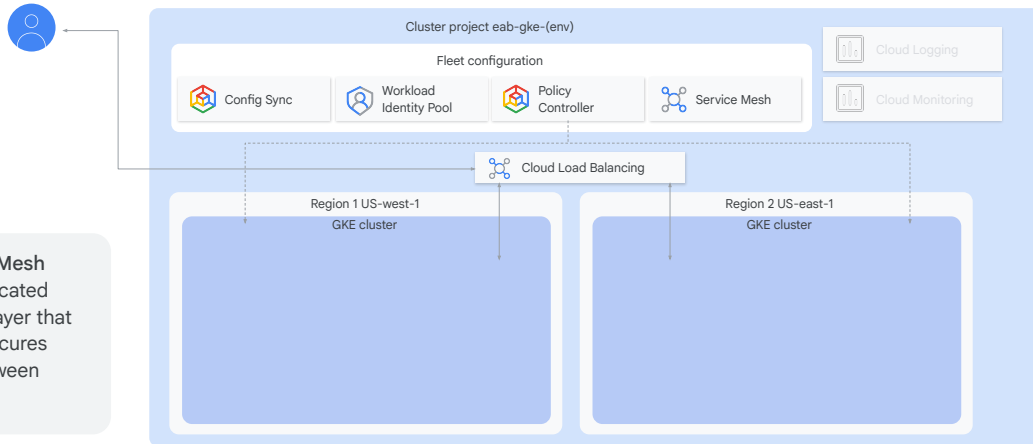
This example displays the production environment, but if there were other environments in the infrastructure like non-production or development environments, they would have the same structure.

Each environment includes resources for fleet configuration and a group of one or more clusters, typically distributed across multiple regions.

In this infrastructure **Cloud Load Balancing** distributes application traffic across regions to Kubernetes service objects. Behind each service is a logical grouping of related pods working together to provide the application's functionality.

Proprietary + Confidential

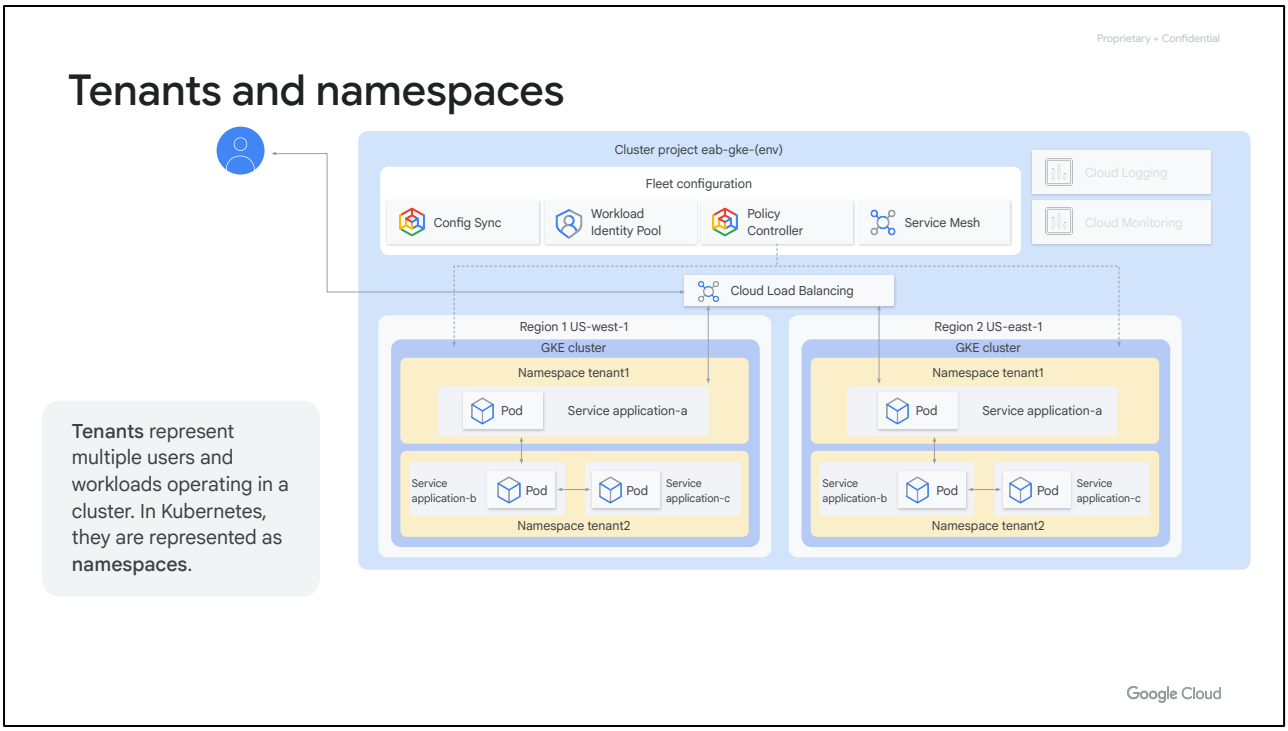
Use Cloud Service Mesh to manage microservices



Cloud Service Mesh provides a dedicated infrastructure layer that controls and secures traffic flow between your services.

Google Cloud

As your Kubernetes environments grow and become more complex, it can be challenging to manage communication between microservices. **Cloud Service Mesh** provides a dedicated infrastructure layer that controls and secures traffic flow between your services, ensuring smooth and reliable communication. Remember that complex environments need resource management.



As your architecture scales another consideration is resource management and optimization. GKE uses a combination of **Namespaces** and **Tenants** to divide and group resources .

Namespaces provide a fundamental way to **divide** a cluster into multiple virtual clusters.

Each namespace can have its own resource quotas and limits, which control the amount of CPU, memory, and other resources that workloads within that namespace can consume.

This allows for resource isolation between different teams or applications.

Tenants in a Kubernetes cluster serve as an abstraction to represent multiple users and workloads, each with different permissions and roles.

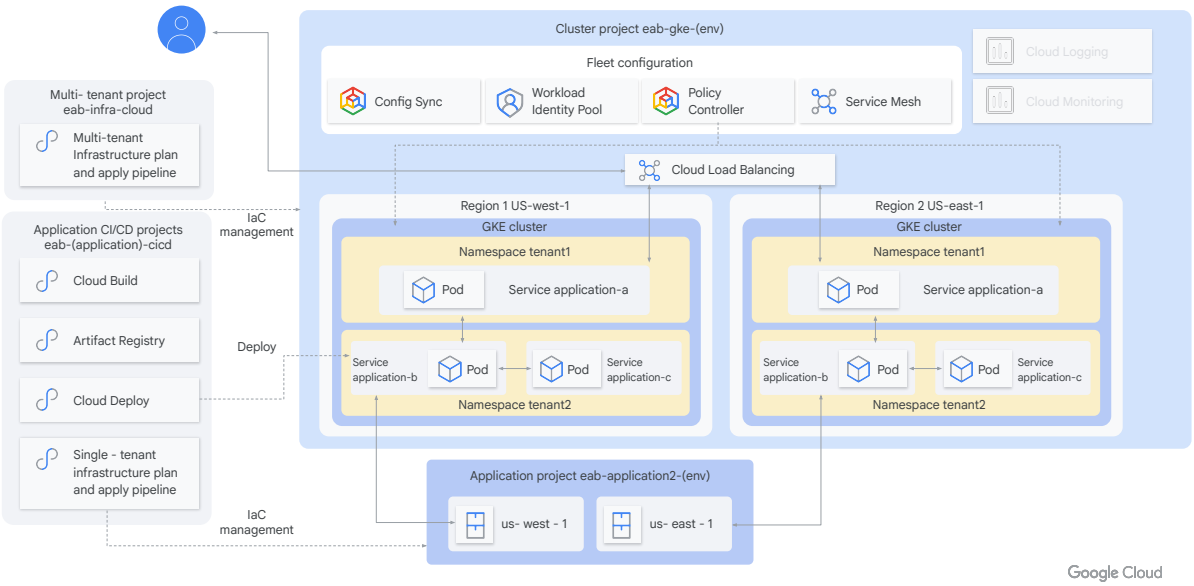
They provide a way to logically **group and isolate services**, ensuring secure multi-tenancy and resource management.

In Kubernetes, tenants are implemented as namespaces.

In this example each cluster has number of tenants for grouping services. There is a tenant pipeline for allocating and balancing resources for tenant request. Namespaces are used to split clusters into workloads .

Proprietary + Confidential

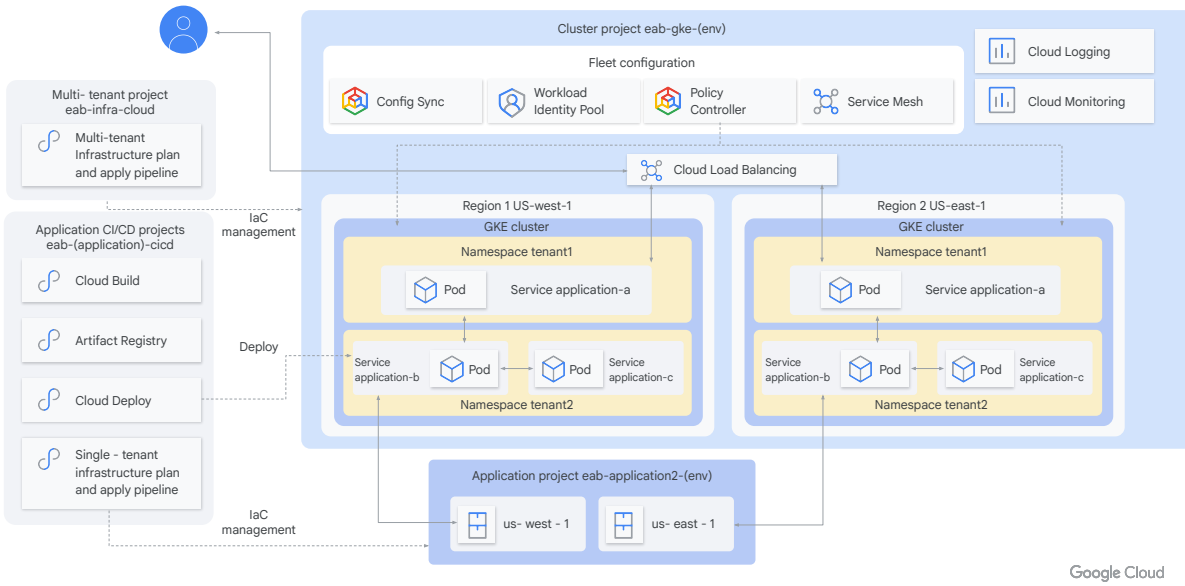
Each Kubernetes service has a dedicated pipeline



Each Kubernetes service is built and deployed using its own dedicated pipeline. That same pipeline is used to deploy the service to the development, non-production, and production environments.

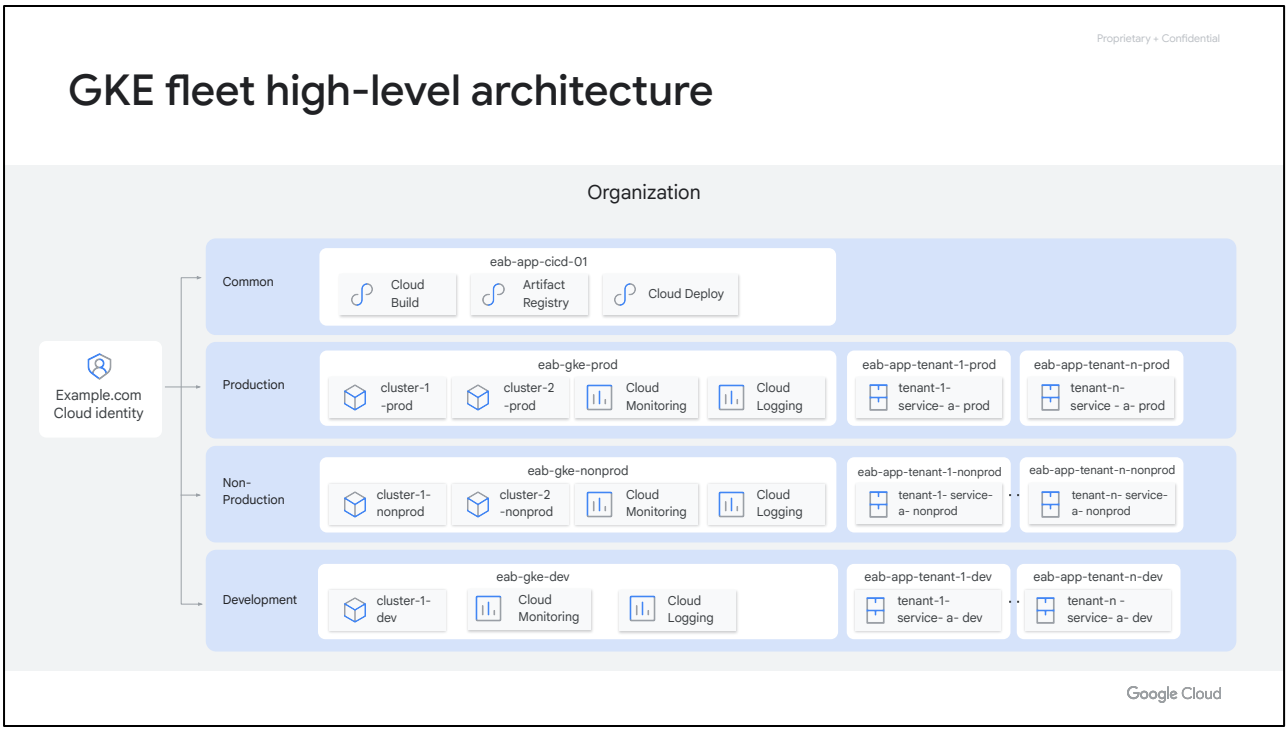
Proprietary + Confidential

Each Kubernetes service has a dedicated pipeline



Other elements are common across all environments, like Cloud Logging and Cloud Monitoring.

The GKE fleet architecture reduces management overhead by reinforcing the concepts of sameness and trust.



When designing an enterprise application, Google Cloud suggests the following architecture blueprint for reference.

You have 4 projects for the infrastructure including the common resources, production, non-production and development.

In the common infrastructure project you host and configure tools like Cloud Build, Cloud Deploy, and Artifact Registry can be used across different environments, providing a consistent way to build, store, and deploy your applications.

In the production, non-production and development infrastructure projects, you have the resources that will be used in each environment. For example, for production you might have 2 clusters to cover high availability and disaster recovery use cases while in development a single cluster for testing might be enough. Additionally, in each environment you will also host adjacent cloud services such as Cloud Monitoring and Cloud Logging. The infrastructure team will be responsible for setting this up.

Next you have your application teams. They will deploy their kubernetes-based applications to a namespace in each cluster, but won't have access to perform any changes in the infrastructure, such as increasing the size of the cluster or creating new clusters. They will be responsible for interacting only with resources within the cluster. That way, you have a clear separation of responsibilities between infrastructure and development teams.

Proprietary + Confidential

Multi-cluster GKE Architecture

01 Centralized cluster management

02 Multi-cluster GKE Architecture

03 Create, connect, and manage GKE fleet clusters

04 Access GKE fleet clusters

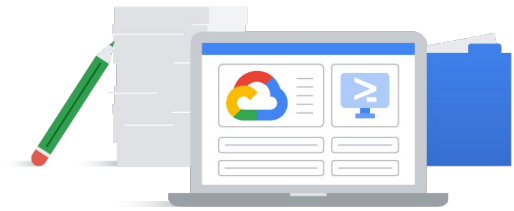
Google Cloud

Next let's explore how to create and manage clusters in a GKE fleet.

Running GKE on VMware or bare metal is beyond the scope of this course. If you are interested in learning more, we recommend the one-day specialization courses.

Proprietary + Confidential

There are three ways to enable GKE



- Google Cloud console
- Cloud Shell
- Terraform

You can enable GKE fleets in the Google Cloud console, by using gcloud commands in Cloud Shell, or with Terraform.

Proprietary + Confidential

Enable GKE fleets by using the Google Cloud console

Google Cloud console

1. Click **Fleet Dashboard.**
2. Click **Create a fleet.** A new fleet is created and clusters are automatically registered. Optionally you can manually configure fleet settings.
3. Change the name of the fleet and edit the cluster list, if desired.
4. Click **Save.**
5. Click **Confirm** to enable APIs and dependencies, create the fleet, and register the clusters.

Google Cloud

The first option is to use the Google Cloud console.

First, navigate to the Google Kubernetes Engine page and then click **Overview**.

Click **Fleet Dashboard**.

Then click **Create a fleet.**

When you enable fleets, a new fleet is created with the same name as your project, and your clusters are automatically registered to it.

You also have the option to manually configure fleet settings.

To change the name of the Fleet, enter the new name on the Fleet registration page. To edit the cluster list, manually select which clusters to include in the fleet.

Click **Save**.

After you click **confirm,** all required APIs and dependencies are enabled, the fleet is created, and specified clusters are registered to the fleet.

Proprietary + Confidential

Use gcloud commands to enable GKE fleets

Enable GKE Hub API

```
gcloud services enable \
--project=PROJECT_ID \
  gkehub.googleapis.com
```

Create an empty fleet

```
gcloud container fleet create \
--display-name=MyAmazingFleet
```

Google Cloud

You can also enable GKE fleets by using gcloud commands.

First, run the `gcloud services enable` command to enable the GKE Hub API. If you haven't set a default project yet, specify your project ID.

To create an empty fleet, run the `gcloud container fleet create` command including the `--display-name` flag that defines the fleet name.

Proprietary + Confidential

Use Terraform to enable the GKE Hub API declaratively

Enable the GKE Hub API in Terraform

```
resource "google_project_service"
"default" {
  project = "your-gcp-project-id"
  service = "gkehub.googleapis.com"

  # Recommended: Prevents Terraform
from disabling the API on destroy
  disable_on_destroy = false
}
```

Google Cloud

As an alternative to gcloud commands, you can use Terraform to enable the GKE Hub API declaratively.

Proprietary + Confidential

Enable services to manage clusters in other clouds

`gcloud services enable \ gkemulticloud.googleapis.com`	Support GKE clusters on multiple platforms.
`gcloud services enable \ gkeconnect.googleapis.com`	Establish connectivity between Google and on-premises clusters.
`gcloud services enable \ connectgateway.googleapis.com`	Securely connect external parties to GKE clusters.
`gcloud services enable \ cloudresourcemanager.googleapis.com`	Interact with Google Cloud metadata

Google Cloud

To manage clusters in other public clouds, you'll need to enable specific Google Cloud services by using the `gcloud services enable` command. Add the googleapis.com domain after each service. For example, `gcloud services enable gkemulticloud.googleapis.com.`

Enable the following services:

- **Gkemulticloud** to support GKE clusters on multiple platforms.
- **Gkeconnect** to establish connectivity between Google and manage clusters both within Google and on-premises.
- **Connectgateway** to securely connect external parties to GKE clusters.
- **Cloudresourcemager** to interact with Google Cloud metadata.

Proprietary + Confidential

Enable services to manage clusters in other clouds

```
gcloud services enable \
logging.googleapis.com

gcloud services enable \
monitoring.googleapis.com
```

Gain insight into cluster health and performance

```
gcloud services enable \
opsconfigmonitoring.googleapis.com

gcloud services enable \
kubernetesmetadata.googleapis.com
```

Extend observability

Google Cloud

- And **Logging and Monitoring** to gain insight into your cluster's health and performance.
- This observability is extended with the **opsconfigmonitoring** and **kubernetesmetadata** APIs to offer more visibility into a multicluster environment.

Proprietary + Confidential

GKE fleet configuration considerations

01 Where will your Kubernetes clusters physically run?

02 Where in Google Cloud will the core management services for your clusters reside?

Google Cloud

Now, when configuring GKE fleets, you'll need to make two key decisions:

First is cluster location. Where will your Kubernetes clusters physically run?

And second is the location of management services, like Policy Controller and Cloud Service Mesh. Where in Google Cloud will the core management services for your clusters reside?

Proprietary + Confidential

Google Cloud management regions

Google Cloud Region	Associated AWS Region	Associated Azure region
europe-west1	eu-central-1 eu-north-1 eu-south-1 eu-west-1 eu-west-2 eu-west-3	francecentral germanywestcentral northeurope uksouth westeurope
us-west1	us-west-1 us-west-2	westus2
us-east4	ca-central-1 us-east-1 us-east-2	canadacentral centralus eastus eastus2 southcentralus
asia-southeast1	ap-east-1 ap-northeast-1 ap-southeast-1	australiaeast japaneast southeastasia
australia-southeast1	ap-southeast-2	australiaeast

Critical cluster data is stored in the Google Cloud region you specify.

Google Cloud regions are associated with other cloud provider's regions for simplified management.

Google Cloud

GKE fleets store your cluster's critical data, including resource information and encryption keys, in the Google Cloud region you specify. This allows you to meet specific data residency and compliance requirements.

Google Cloud regions are associated with other cloud provider's regions for simplified management.

For example, if you plan to run your cluster in AWS, but your data is located in Asia, you could choose the asia-southeast1 region in Google Cloud, and the ap-southeast-1 region in AWS.

More information on running clusters in other platforms is available in the reading materials for this module.

Proprietary + Confidential

Multi-cluster GKE Architecture

01 Centralized cluster management

02 Multi-cluster GKE Architecture

03 Create, connect, and manage GKE fleet clusters

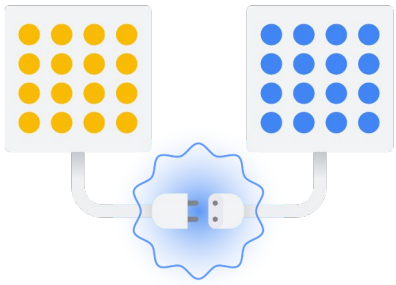
04 Access GKE fleet clusters

Google Cloud

One of the key features of GKE is providing multi-cluster management capabilities that allow you to manage Kubernetes clusters running in various environments.

Proprietary + Confidential

Connect Agent



- A Kubernetes Deployment that runs in your GKE cluster, handles Kubernetes API requests, and establishes a secure connection to Google Cloud
- The connection can pass through any network devices and it doesn't require you to have a public IP address in your cluster.
- Enables clusters to communicate with the control plane. User interactions are recorded in Kubernetes Audit Logs for security and auditing purposes.

Google Cloud

Connect Agent is a Kubernetes Deployment that runs in your GKE cluster, handles Kubernetes API requests, and establishes a secure connection to Google Cloud. This connection is authenticated and encrypted using Transport Layer Security (TLS).

The connection can pass through any network devices like NATs and firewalls, and it doesn't require you to have a public IP address in your cluster.

After the cluster is connected, it can communicate with the control plane in Google Cloud. User interactions are recorded in Kubernetes Audit Logs for security and auditing purposes.

Proprietary + Confidential

To manage clusters, register them with a GKE fleet

```
$ gcloud container fleet \
memberships register $M
```

- Registers a cluster with a GKE fleet.
- Installs and configures Connect Agent.

Control and manage clusters:

- Google Cloud console
- Cloud Shell

Google Cloud

Connect Agent is only required if your GKE clusters are running in cloud providers outside of Google Cloud.

If you want to manage an existing Kubernetes cluster with GKE, you must register it with GKE fleet.

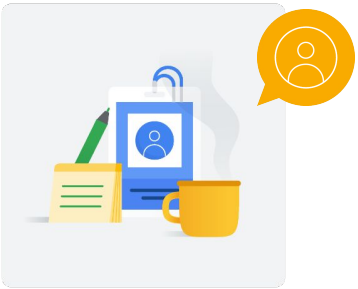
This allows you to access and control all your clusters from a central location in the Google Cloud console.

To register a cluster, you can use the gcloud container fleet memberships register command, which installs and configures the Connect Agent in your cluster.

Then you can use the Google Cloud console and the gcloud command-line tool to control and manage your registered clusters.

Proprietary + Confidential

Log in to Google Cloud and manage your clusters



To login to the Google Cloud console, you can use:

- Your Google identity
- Third-party identity providers
- Bearer tokens

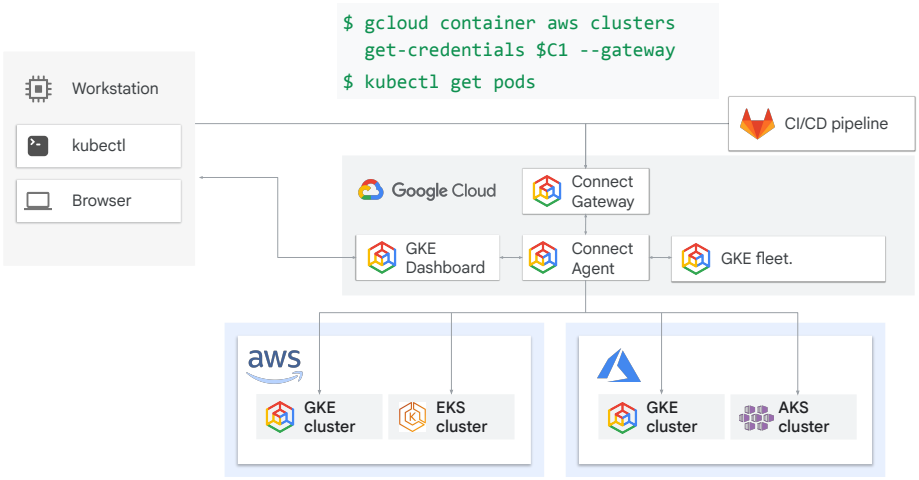
Google Cloud

To log in and access clusters, you can use several different types of credentials, including:

- Google identity.
- Third-party identity providers like GKE Identity Service with OIDC, Microsoft Entra ID (Azure AD), or Okta.
- And bearer tokens.

Proprietary + Confidential

Connect Gateway



Google Cloud

Connect Gateway provides a secure entry point for accessing your registered clusters from anywhere, regardless of your network location.

To access your registered clusters running in AWS or Azure, you can use the `gcloud container aws (or azure) clusters get-credentials` command. The Connect Agent securely forwards your requests to the Google Cloud API server.

With GKE's multi-cluster management, you have a centralized location for managing a multi-cluster environment. This enables you to create a unified CI/CD process across clusters in different environments and networks.